PROBLEM SOLVING AND DEBUGGING

CS1910

Computer Science I

EXAMPLE CODE

- https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/10-Problem-Solving-Debugging/CS1910_Problem-Solving-Debugging_Workbook.ipynb

PROBLEM SOLVING



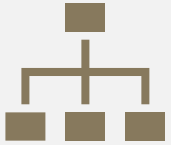
Computer Science is fundamentally about solving problems



We use computers and programming to automate solutions



How do we get from a problem idea to working code



A structured problem-solving process

4 STAGE PROBLEM SOLVING MODEL

Analyze the Problem
(Understand)

Generate Alternatives
(Design)

Implement
(Coding)

Common mistake: Making this the starting point

Evaluate
(Testing and Validation)

Recall

COMPUTATIONAL THINKING

Aspects of computational thinking apply directly to steps in the problem solving process

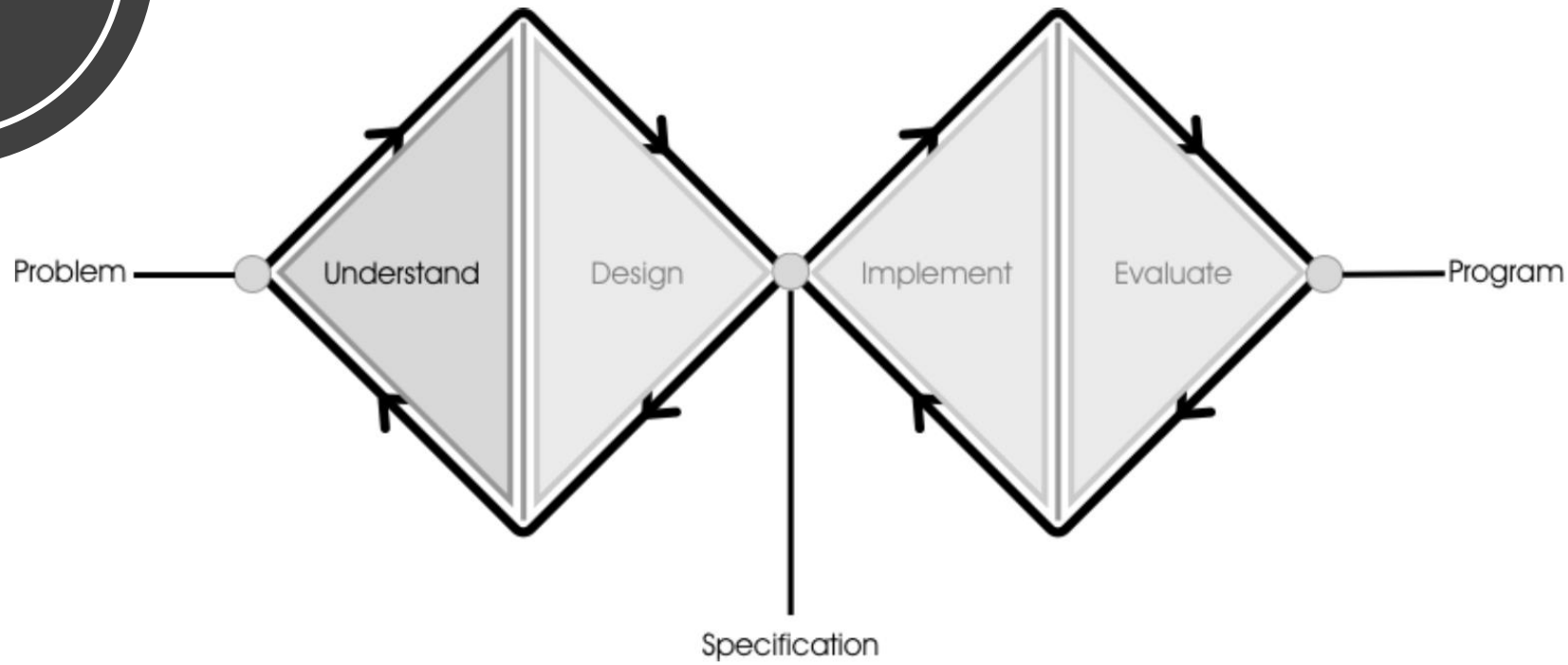
Computational Thinking

- Decompose
- Pattern Recognition
- Abstraction
- Algorithms
- Validation



- Understand (Analysis)
- Design (Generate alternatives)
- Implement (Coding)
- Evaluate (Testing and Validation)

DOUBLE DIAMOND



UNDERSTAND

- Asking questions to improve your understanding of the problem
  - What is the desired outcome? (*requirements*)
  - What information do I have that I can use?
  - What information do I need?
    - Can you develop your own examples of inputs and their corresponding output?
    - Can you simulate or hand calculate simple cases?
  - Are there special cases?
  - Is there redundant information?
  - Are any parts of the problem contradictory?
  - Are there parts of the problem that are ambiguous?
  - Can we break the problem into parts?

DESIGN

- This is where you create/find alternatives for solving your problem. These are questions that point to tactics
  - Have you solved a problem like this before?
  - Has anyone else solved a problem like this?
  - Are there more general problems that are similar
  - Are there more specific problems that are similar
  - Can we develop pseudo code or flow charts
- We *iterate* on these questions to generate a pool of potential solutions
- You may discover that you do not understand everything and need to return to the understanding step to broaden your perspective
- Once you have a list of potential solutions you ask a different set of questions of each solution:
  - What assumptions am I making?
  - Can I break this problem into smaller parts?
  - What data do I need and how will I store it?
  - Which design is easiest to maintain?
  - How do the designs differ in clarity, time, and space?
  - Does the solution scale to very large or very small values?
- The answers to the above help you choose a solution that has the most benefits to solving your problem
- That chosen solution becomes your *specification*

FIND A PARTNER

- Recall our 1 on 1 basketball problem
- A game goes up to 11 points, the first person to 11 wins (but you must win by 2)
- The scoring in the game is specified in the order that the points were scored between player A and B, each basket is worth 1 or 2 points
  - Example: "A2B2A2A2A1A1A1A2"
- We want to know who won the game: A or B
- Most students in the class designed a solution that required variables to count each person's scores along the whole of the game
  - Consider the left half of the diamond:
    - **Understand and Design**

FIND A PARTNER

- Understanding the basketball problem:
  - What are we being asked?
  - What is redundant?
  - Can we decompose the problem into smaller aspects?
  - Did you try building other examples, other than what was given?
- Designing a solution:
  - Decompose into smaller tasks
  - How to store or represent data
  - Existing solutions or patterns?
  - Consider pros and cons

IMPLEMENT

- Coding may uncover issues with the design
  - What part of the design is unclear once I start coding?
  - Did I discover a missing step, condition, or data?
- Be prepared to revisit your solution
  - If something feels hard to implement, can I simplify the design (refactoring)?
  - What is the smallest change to make the code and design align?
- Consider efficiency
  - Am I doing unnecessary work (repeated loops, repeated calculations)?
  - Is there a clearer data structure that reduces steps? (CS2920)
  - Does the code align with the solution?
  - Do function names and variable names reflect the steps in my plan?
  - Does each function do one clear job from the design? (Cohesion)
- Practical checks while coding
  - Can I run and test each small piece right after I write it?
  - Do I have simple, normal, and edge-case inputs to try quickly?
  - Is my code readable by someone else or by me in the future?
  - Is duplicated code extracted into a helper?

IMPLEMENTATION

(HERE IS ONE I DID EARLIER!)

```
def get_winner(scoring_sequence):
    score_a = 0
    score_b = 0
    index = 0

    while index < len(scoring_sequence):
        current_player = scoring_sequence[index]
        points_scored = int(scoring_sequence[index + 1])

        if current_player == "A":
            score_a = score_a + points_scored
        else:
            score_b = score_b + points_scored

        # BUG: stops as soon as someone reaches 11 (does not check 2-point lead)
        if score_a >= 11 or score_b >= 11:
            if score_a > score_b:
                return "A"
            else:
                return "B"

        index = index + 2

    # End of sequence: pick the higher score
    if score_a > score_b:
        return "A"
    else:
        return "B"
```

EVALUATION

- Validation
  - Does the program solve the problem I was asked to solve?
  - Does the program meet the *requirements*?
- Correctness
  - Does the program produce the expected output for normal cases?
  - Do I have at least: 1 normal case, 1 edge case, 1 error case?
  - What edge cases might break it (empty input, maximum size, duplicates)?
- Robustness
  - Does the program handle bad or unexpected input gracefully?
  - Are error messages clear and helpful?

EVALUATION TEST CASES

- Normal Cases: Happen frequently
  - Normal Case 1: A player wins on last basket going from 10 to 11 and the other player is at 9 or lower (win by 2)
    - Does not matter if it is 11-0, 11-5, or 11-9, all are examples of that case – Can have A1 repeated 11 times
  - Normal Case 2: A player wins on last basket going from 10 to 12 and the other players is at 10 or below
    - Does not matter if it is 12-0, 12-5 or 12-10, all are examples of the same case. 0 Can have B2 repeated 6 times.
- Edge Cases: Happen seldomly or in specific situations
  - Edge case: Extended play (called a deuce like in tennis):
    - Score of A-B is10–10, then a player A scores 1 point. Score is now 11-10.
    - Alternatives of test cases (Design of test cases)
      - A scores 1 more point (A1) – A wins
      - B scores 3 more points one at a time (B1B1B1) – B wins
      - B scores 1 point and then 2 points (B1B2) – B wins
      - B scores 1, creating a tie, and then we are back in another example of a tie

TEST CASES TRANSLATE TO PYTHON

```
# Build the test data
    # 11–0 for A
    normal_case_a = "A1" * 11
    # 12–0 for B
    normal_case_b = "B2" * 6
    # 10–10 → 11–10 → 11–13 (B should win)
    deuce_case = ("A1B1" * 10) + "A1" + "B1B1B1"

    #A win with 11 points
    print("Buggy on normal_case_a? (expect A):", get_winner(normal_case_a))
    #B wins with 12 points
    print("Buggy on normal_case_b? (expect B):", get_winner(normal_case_b))
    #B wins with 13 points
    print("Buggy on deuce_case? (expect B):", get_winner(deuce_case))  # Prints A!
```

TEST CASES
RUN THE CASES

```
# Build the test data
    # 11–0 for A
    normal_case_a = "A1" * 11
    # 12–0 for B
    normal_case_b = "B2" * 6
    # 10–10 → 11–10 → 11–13 (B should win)
    deuce_case = ("A1B1" * 10) + "A1" + "B1B1B1"

    #A win with 11 points
    print("Buggy on normal_case_a? (expect A):",
    get_winner(normal_case_a))
    #B wins with 12 points
    print("Buggy on normal_case_b? (expect B):",
    get_winner(normal_case_b))
    #B wins with 13 points
    print("Buggy on deuce_case? (expect B):",
    get_winner(deuce_case))  # Prints A!
```

Buggy on normal_case_a (expect A): A

Buggy on normal_case_b (expect B): B

Buggy on deuce_case (expect B): A

- Oh no! Our last test case has failed!
- Now what?
- Well .. we have a bug … so now we need to debug our code
- This is returning from Evaluation and iterating back to Implementation
- It is possible that we didn't understand the problem and have to go back to the beginning – but let's see if we can figure out where we went wrong

TYPES OF ERRORS

- **Syntax errors**
  - Identified by the interpreter
    - Example: a forgotten ':' at the end of a function declaration
  - Experience helps us interpret and rectify the syntax errors produced by the interpreter

```
def divide_by(x,y)
   return x/y
print(divide_by(3,4))
print(divide_by(10,0))
```

```
Cell In[5], line 1
        def get_winner(scoring_sequence)
                                        ^
SyntaxError: expected ':'
```

RUNTIME ERROR

- **Runtime Errors**
  - Occur when something goes wrong while the program is running
  - Typically come with a stack trace and a message indicating the error
  - Picture a stack of plates:
    - Each time a function is called the system state is saved including the line number to return and puts it's "plate" on top of the last function call in the stack
    - When a function finishes, it returns to the line it was called from and removes itself from the top of the stack of plates
  - When a run time error happens it dumps out all of the plates starting at the bottom most plate for you to see what was called so you can identify where it had the error happen

```
def divide_by(x,y):
  return x / y

print(divide_by(10,0) )
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
/tmp/ipython-input-654804771.py in <cell line: 0>()
      2   return x / y
      3
----> 4 print(divide_by(10,0) )

/tmp/ipython-input-654804771.py in divide_by(x, y)
      1 def divide_by(x,y):
----> 2   return x / y
      3
      4 print(divide_by(10,0) )

ZeroDivisionError: division by zero
```

SEMANTIC ERROR

- **We currently have a semantic errors**
  - The program produces the incorrect results
  - Can be difficult to track down
- The program runs (no syntax problems), runs all the way to the end (no errors thrown at run time), but produces the wrong result and we have no further information to help us

DEBUGGING

- In all error cases, we need to **debug** our program:
  - Syntax error: Fix the syntax
  - Runtime and Semantic: Find out what is happening that is going wrong
- Debugging is a reactive process to a fault in a program
  - Identify the reason for the fault
  - Repair the fault
- The complexity of this process is varied
  - For small programs it might be quite simple
  - For larger programs the process may be very complex

DEBUGGING TECHNIQUES

- Debugging by inspection
  - Sometimes we can look at the code and spot an issue
  - Print out values that we suspect should be something and determine why they might not be what we expect
  - Often, we have an error because some data was computed or stored incorrectly at an earlier point and by printing it out, we can determine where (and hopefully why) it has deviated
  - Can be successful – but if you find that you have tried it, and not found useful information, it can turn into guesswork that is just wasting time
- Hand tracing code
  - Simulate the action of the code on paper, track the variables and their expected values and compare against what we are seeing
- Integrated Debuggers
  - Programs that allow us to inspect the "state" of the program while it is running
    - Pause the program at certain lines and advance line by line through the code

DEBUGGING TECHNIQUES

- Debugging by inspection
  - Sometimes we can look at the code and spot an issue
  - Print out values that we suspect should be something and determine why they might not be what we expect
  - Often times we have an error because some data was computed or stored incorrectly at an earlier point and by printing it out we can determine where (and hopefully why) it has deviated
- Hand tracing code
  - Simulate the action of the code on paper, track the variables and their expected values and compare against what we are seeing
- Integrated Debuggers
  - Programs that allow us to inspect the "state" of the program while it is running
    - Pause the program at certain lines and advance line by line through the code

DEBUGGING BY PRINTING

```
#Print off information each time scoring is recorded to
    see sequence
    print("Event:", current_player + str(points_scored),
          "| Before:", before_a, "-", before_b,
          "| After:", score_a, "-", score_b)

    # Watch the first time someone hits 11+
    if score_a >= 11 or score_b >= 11:
        lead = score_a - score_b if score_a >= score_b else
    score_b - score_a
        print("Reached 11+; lead is", lead, "→ returning
    NOW (this is the bug)")
        if score_a > score_b:
            return "A"
        else:
            return "B"
```

IDENTIFY THE BUG

- Here is the printout – we have learned that the problem (as suspected) is happening when someone has 11-10 for the first time.

Event: A1 | Before: 0 - 0 | After: 1 - 0

Event: B1 | Before: 1 - 0 | After: 1 - 1

Event: A1 | Before: 1 - 1 | After: 2 - 1

Event: B1 | Before: 2 - 1 | After: 2 - 2

Event: A1 | Before: 2 - 2 | After: 3 - 2

Event: B1 | Before: 3 - 2 | After: 3 - 3

Event: A1 | Before: 3 - 3 | After: 4 - 3

Event: B1 | Before: 4 - 3 | After: 4 - 4

Event: A1 | Before: 4 - 4 | After: 5 - 4

Event: B1 | Before: 5 - 4 | After: 5 - 5

Event: A1 | Before: 5 - 5 | After: 6 - 5

Event: B1 | Before: 6 - 5 | After: 6 - 6

Event: A1 | Before: 6 - 6 | After: 7 - 6

Event: B1 | Before: 7 - 6 | After: 7 - 7

Event: A1 | Before: 7 - 7 | After: 8 - 7

Event: B1 | Before: 8 - 7 | After: 8 - 8

Event: A1 | Before: 8 - 8 | After: 9 - 8

Event: B1 | Before: 9 - 8 | After: 9 - 9

Event: A1 | Before: 9 - 9 | After: 10 - 9

Event: B1 | Before: 10 - 9 | After: 10 - 10

Event: A1 | Before: 10 - 10 | After: 11 - 10

Reached 11+; lead is 1 → returning NOW (this is the bug)

CORRECT THE BUG

```
def get_winner_fixed(scoring_sequence):
    score_a = 0
    score_b = 0
    index = 0

    while index < len(scoring_sequence):
        current_player = scoring_sequence[index]
        points_scored = int(scoring_sequence[index + 1])

        if current_player == "A":
            score_a = score_a + points_scored
        else:
            score_b = score_b + points_scored

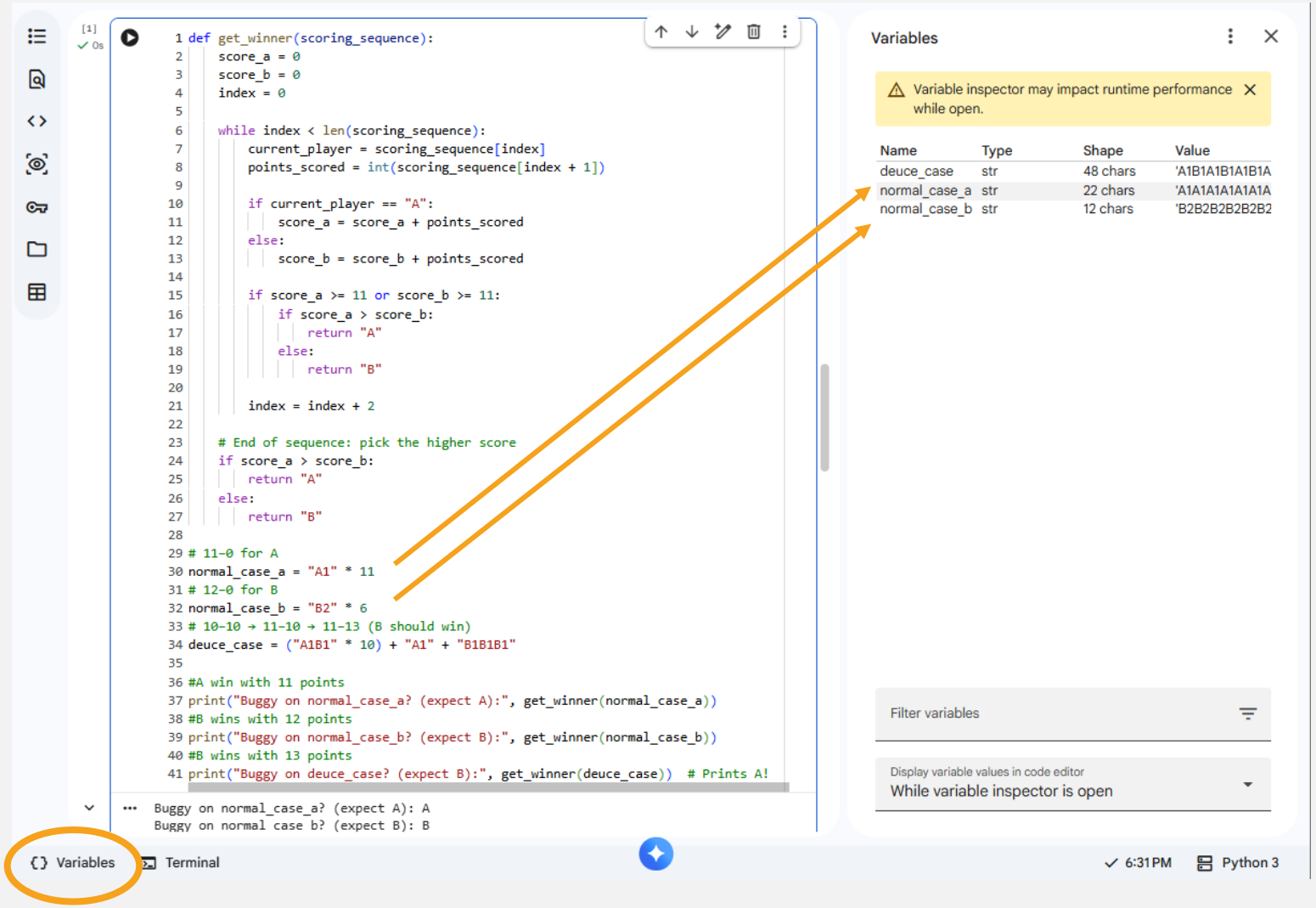
        # Logic correction: must reach at least 11 AND lead by 2
        lead_for_a = score_a - score_b
        lead_for_b = score_b - score_a
        if (score_a >= 11 or score_b >= 11) and (lead_for_a >= 2 or
lead_for_b >= 2):
            if score_a > score_b:
                return "A"
            else:
                return "B"

        index = index + 2

    # If no 2-point lead by 11 occurred in the sequence, choose the
    higher score
    if score_a > score_b:
        return "A"
    else:
        return "B"
```

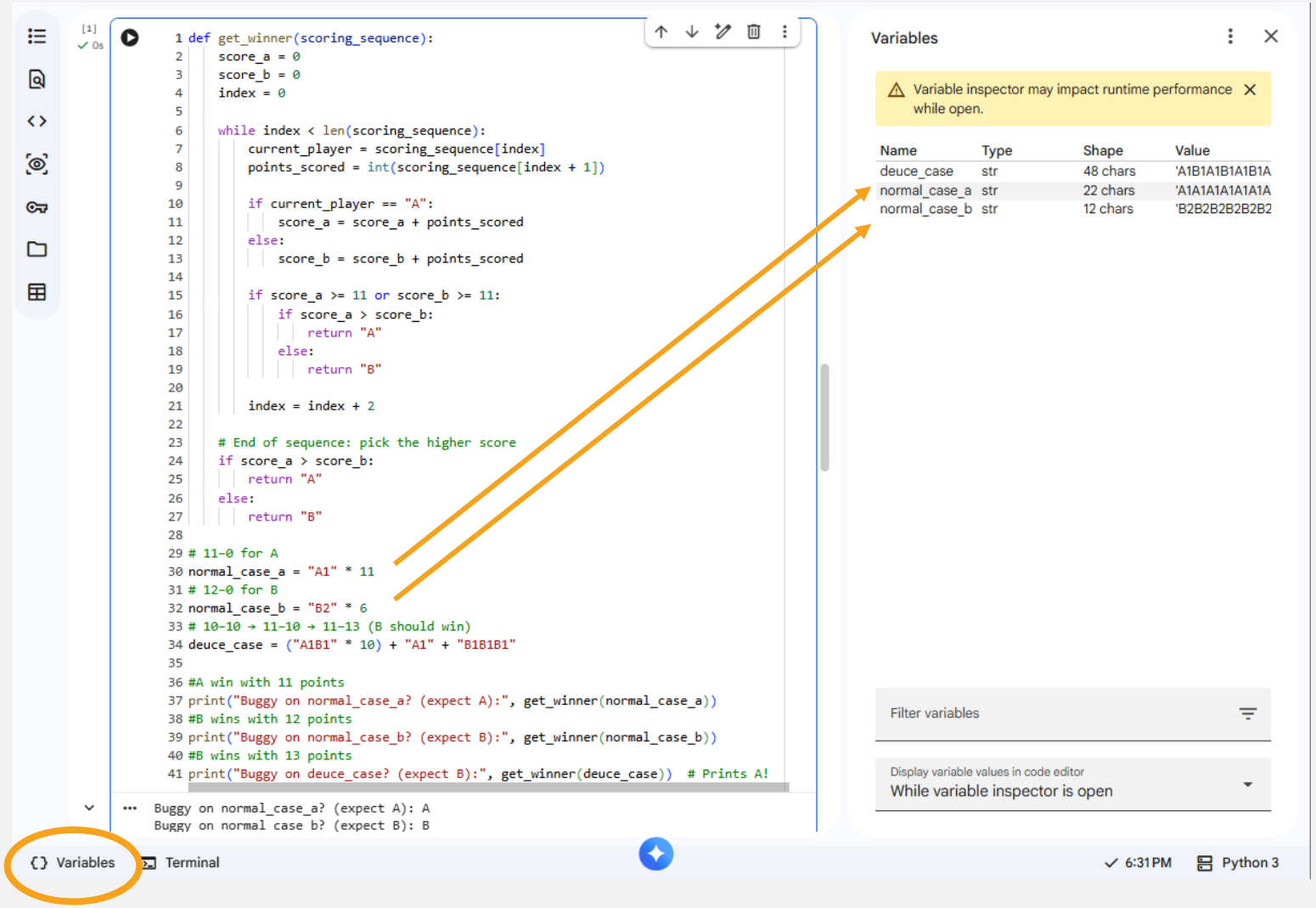
GOOGLE COLAB STATE INFORMATION

- Colab comes equipped with a variable inspector
- This allows you to inspect the state of variables as you run your program
- If a program pauses, then you can see the variables as they are populated at that moment



GOOGLE COLAB STATE INFORMATION

- Colab comes equipped with a variable inspector
- This allows you to inspect the state of variables as you run your program
- If a program pauses, then you can see the variables as they are populated at that moment



```
#consider the below code that contains an error
input_string = "A banana"

#convert the string to CAPITAL LETTERS
input_string.capitalize()

#count all the capital letter A occurrences
count_A = 0
for char in input_string:

    if char == "A":
        count_A += 1

#how many letter a's did we find
print(count_A)
```

```
1
```

PARTNER WORK

- Find a partner
- Practice showing how to find a bug
- Print out variables to showcase where the error occurs
- Open the Variable tab to see if that helps diagnose the issue
- Goal: practice using these methods and show how they might help

CLASS PARTICIPATION

- In the partner work example was the error:
  - Runtime Error
  - Semantic Error
  - Syntax Error
  - Gross Error

DEBUGGER

- A program that allows you to inspect and control the execution of your program is called a debugger
- Most programming environments come with a debugger built in
- Common debugger commands:
  - Pause the program at a particular line of code (breakpoint)
  - Navigate through the code manually
  - Display values of a particular variable or other information

COLAB DEBUGGER

- Some debuggers have nice graphical interfaces (buttons, etc)
- Colab has a basic debugger provided through the python environment (pdb = python debugger)
- For this course we only use a couple commands:
- breakpoint() (pause the program at a particular point)
- print(…) print out information
- c = continue to the next breakpoint or program end
- q = quit the debugger

DEBUGGER

```
number1 = input("Enter a number: ")
breakpoint()
number2 = input("Enter a number: ")

breakpoint()
sum_numbers = number1 + number2

breakpoint()

print(f"the sum is {sum_numbers}")
```

Program will pause at this line and enter debug mode

Type c to continue to next breakpoint
Or print out data

PDB DEMO

TRY AT HOME

- Ensure if you re-wrote the midterm you would be able to correctly answer all questions

SELF-CHECK

- Consider the following code:

```
#Bug Code Example
    number1 = input("Enter a number:")
    number2 = input("Enter another number:")

    sum_numbers = number1+number2
    print(f"Sum of numbers: {sum_numbers}")
```

What type of error will I encounter?

Think about your test data …

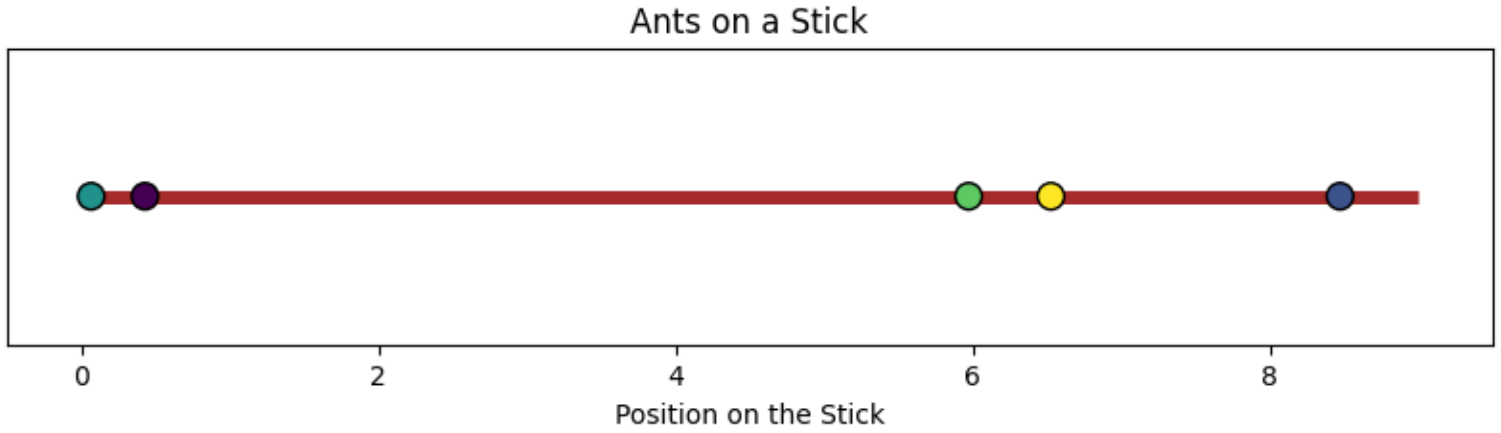


ADDITIONAL CASE STUDY

MORE FIND A PARTNER: THE ANTS GO MARCHING

- Consider that ants march on a stick
- They travel in one direction but if they bump into another ant both ants reverse their direction and walk the other way
- If they reach the end of the stick they fall off
- Ants travel at 1cm/s and the stick is 9 cm long
- What is the minimum or maximum amount of time until the stick has no ants on it

- Input: the number of ants on the stick and the starting location for each ant

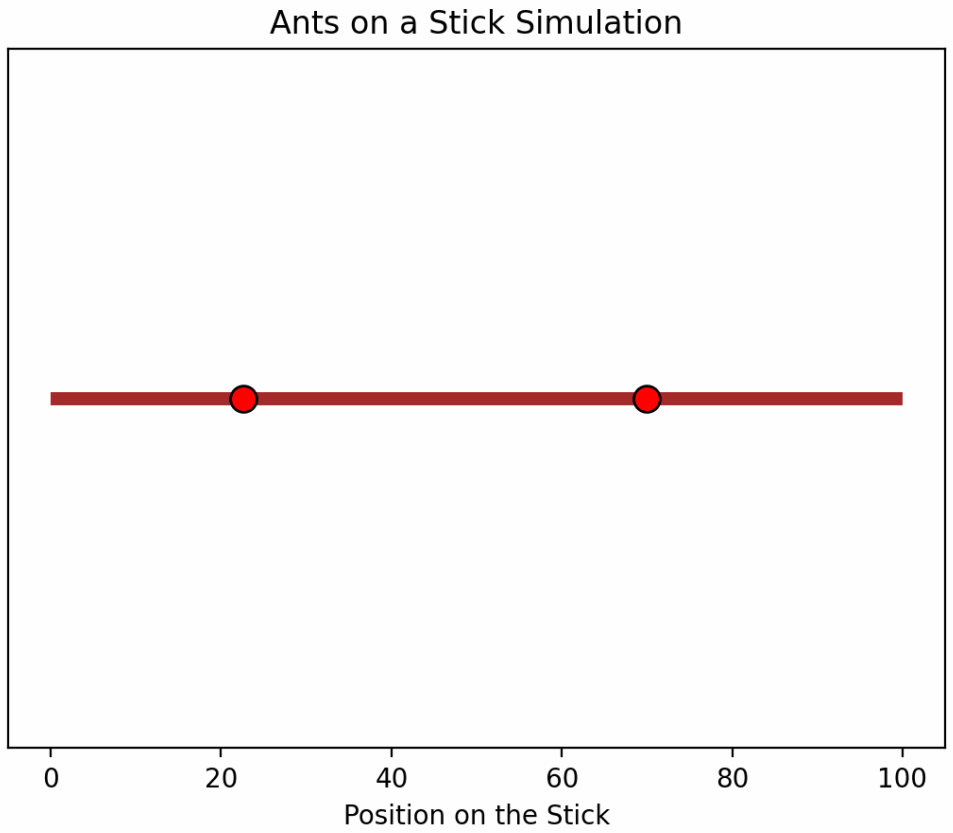


ANTS GO MARCHING

Input: the number of ants on the stick and the starting location for each ant

Expected output: The minimum and maximum possible time for the stick to empty

ANTS ON A STICK



TRY AT HOME

- Parity
  - In Computer Science a bit is something that has 2 states (on and off). We represent bits with either 0 or 1
  - A bit string is a string containing only 0s and 1s e.g., "0100110"
  - Parity is a count of the number of 1's in a bit string if there is an even number we say the bit string has even parity.
  - Suppose we always like our bit strings to have even parity.
  - Given a bit string your task is to concatenate a character to the end of the bit string but ensure the parity of the bit string you create is always even

The solution is not the goal here. Follow the problem-solving process